

Real-Time Credit Card Fraud Detection System

A machine learning–based fraud detection application for identifying potentially fraudulent credit card transactions through a real-time prediction API and interactive web interface.

The project combines **data preprocessing, class-imbalance handling, model comparison, hyperparameter tuning, threshold optimization, explainable AI, FastAPI deployment, and Streamlit visualization** into an end-to-end fraud detection system.

The final system uses a **tuned XGBoost classifier**, an optimized classification threshold, **SHAP-based feature contributions**, and an optional **LLM explanation layer** that converts model-derived information into concise plain-English explanations.

This project was developed as part of the **3MTT Data Science Mentorship Capstone**.

Table of Contents

Project Overview.....	4
Problem Statement	5
System Objectives	6
Project Architecture	7
Dataset	8
Data Preprocessing	9
Class Imbalance Handling	10
Models Evaluated.....	11
Model Comparison	11
Hyperparameter Tuning.....	12
Cross-Validation.....	13
Classification Threshold Optimization	13
Final Model.....	14
Model Serialization	15
Explainability	16
LLM Explanation Layer.....	16
FastAPI Backend.....	18
API Input Validation	19
Prediction Pipeline	20
Streamlit Frontend	21
Deployment Consistency	22
Performance Testing.....	23
Error Handling	23
Project Structure.....	24
Technologies Used	26
Installation and Setup	27
Limitations	28

Future Improvements	29
Conclusion.....	30
Author	31

Project Overview

Credit card fraud detection is a highly imbalanced binary classification problem because fraudulent transactions represent only a very small proportion of all transactions.

This project develops an end-to-end fraud detection system that:

- preprocesses transaction data consistently;
 - prevents preprocessing leakage;
 - evaluates approaches to class imbalance;
 - compares multiple machine learning algorithms;
 - tunes the selected model using cross-validation;
 - evaluates models using fraud-relevant metrics;
 - optimizes the classification threshold;
 - serializes the final model and preprocessing artifacts;
 - provides individual transaction predictions;
 - supports CSV-based batch prediction through the frontend;
 - provides SHAP-based model explanations;
 - optionally generates plain-English explanations through an LLM;
 - exposes predictions through a FastAPI REST API; and
 - provides an interactive Streamlit frontend.
-

Problem Statement

Fraud detection presents a challenging machine learning problem because the fraudulent class is extremely rare compared with legitimate transactions.

A model can achieve high overall accuracy while still failing to identify fraudulent transactions. Therefore, this project focuses on metrics that better represent minority-class performance:

- Precision
- Recall
- F1 Score
- ROC-AUC
- PR-AUC
- Confusion Matrix

The objective is to develop a model that provides a useful balance between detecting fraudulent transactions and limiting unnecessary fraud alerts.

System Objectives

The project was designed around five major objectives.

1. Build a reliable machine learning pipeline

Develop reusable components for:

- data preparation;
- sampling;
- model training; and
- model evaluation.

2. Compare and select an appropriate model

Evaluate multiple classification algorithms and identify a strong final candidate based on fraud-relevant performance.

3. Optimize the final model

Apply:

- hyperparameter tuning;
- stratified cross-validation; and
- classification threshold optimization.

4. Provide model explainability

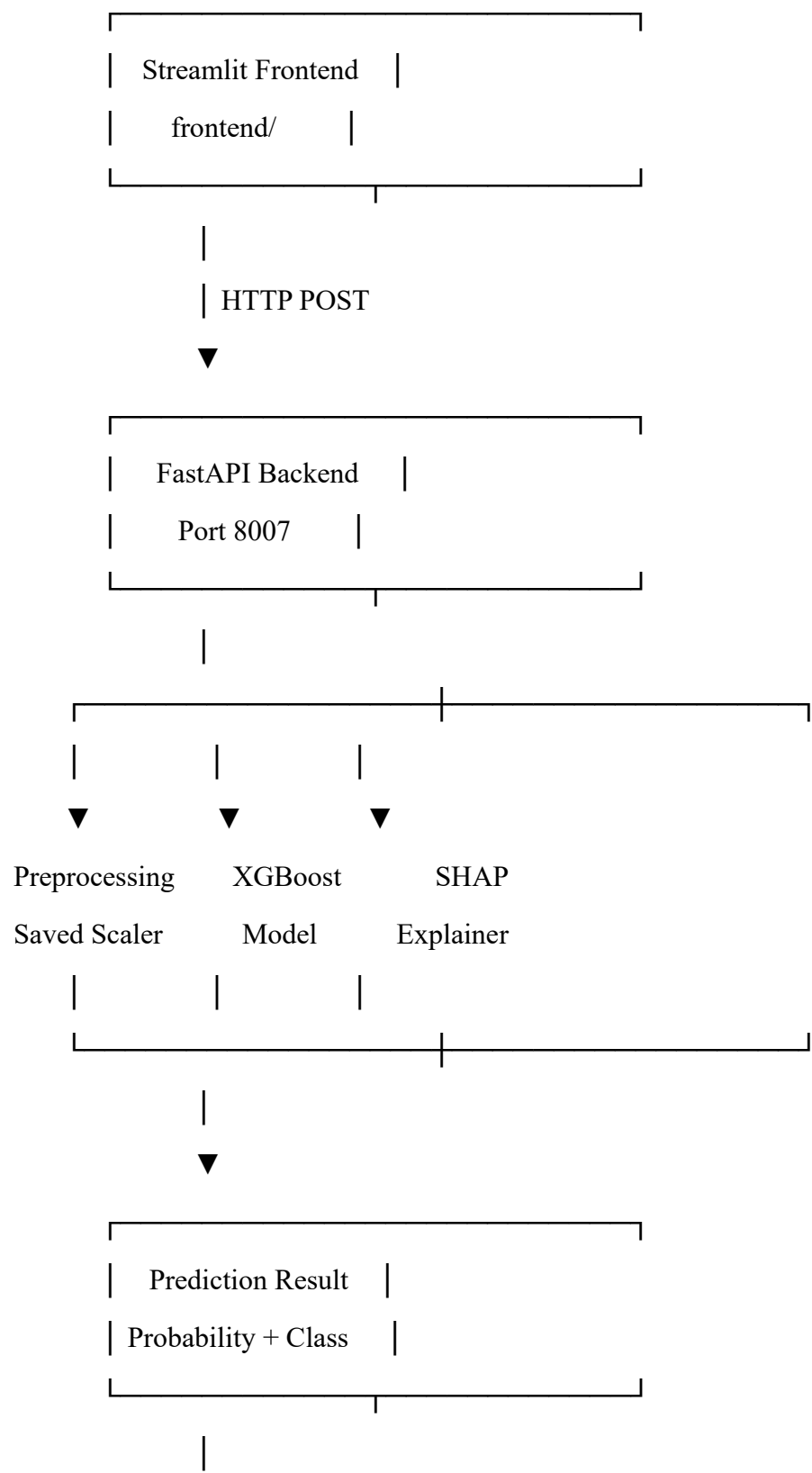
Use SHAP to identify the features contributing most strongly to individual predictions.

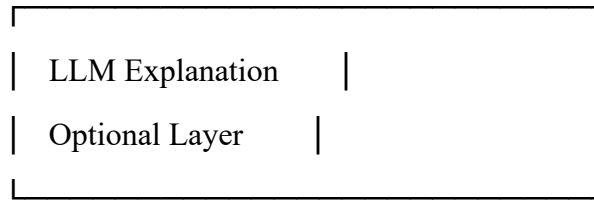
5. Deploy the model as an application

Provide:

- a FastAPI prediction service;
 - a Streamlit frontend;
 - manual transaction prediction;
 - CSV-based prediction; and
 - human-readable explanations.
-

Project Architecture





The **XGBoost** model makes the fraud classification decision.

SHAP provides model-derived feature contributions.

The optional LLM is used only to convert the supplied model information into plain language. It does not make an independent fraud decision.

Dataset

The project uses the **ULB/Kaggle Credit Card Fraud Detection** dataset.

Dataset characteristics

Property	Value
Transactions	284,807
Predictor features	30
Target	Class
Legitimate transactions	284,315
Fraudulent transactions	492
Missing values	None

Target encoding:

Class = 0 → Legitimate transaction

Class = 1 → Fraudulent transaction

The predictor variables are:

Time

V1–V28

Amount

The V1–V28 variables are anonymized features.

Because the dataset is severely imbalanced, accuracy is not used as the primary model-selection criterion.

Data Preprocessing

Reusable preprocessing functions are implemented in:

`src/data_prep.py`

The preprocessing workflow includes:

1. Dataset loading
2. Feature and target separation
3. Stratified train-test splitting
4. Standardization of Time and Amount

Train-Test Split

The dataset was divided using an 80/20 stratified split:

Training data → 80%

Testing data → 20%

Configuration:

`random_state = 42`

`stratify = y`

Stratification preserves the minority-class proportion between the training and testing sets.

Feature Scaling

Only the following features are standardized:

Time

Amount

The StandardScaler is fitted **only on the training data**.

The fitted scaler is then applied to:

- training data;
- test data; and
- incoming API transactions.

This prevents test-set information from leaking into the preprocessing process.

The fitted scaler is serialized and reused by the FastAPI backend to maintain consistency between model development and deployment.

Class Imbalance Handling

The project evaluated several approaches to the severe class imbalance:

- class weighting;
- Random Undersampling; and
- SMOTE.

The sampling implementations are contained in:

src/sampling.py

Sampling Experiments

Random Undersampling and SMOTE were evaluated using XGBoost against the same untouched test set.

Strategy	Precision Recall F1			ROC-AUC	PR-AUC
Random Undersampling	0.0441	0.9082	0.0842	0.9758	0.6987
SMOTE	0.3468	0.8776	0.4971	0.9759	0.8481

SMOTE substantially outperformed random undersampling on precision, F1 score, and PR-AUC in this experiment.

However, the **final deployed XGBoost model was trained using class weighting through `scale_pos_weight`**, rather than the SMOTE-trained model.

Models Evaluated

The project evaluated the following model configurations:

1. Logistic Regression
2. Random Forest
3. XGBoost Baseline
4. CatBoost
5. Tuned XGBoost

Training implementations are included in:

src/train.py

Model Comparison

The main model comparison was performed using the same held-out test set.

Model	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Logistic Regression	0.0610	0.9184	0.1144	0.9722	0.7159
Random Forest	0.9059	0.7857	0.8415	0.9573	0.8629
XGBoost Baseline	0.7810	0.8367	0.8079	0.9685	0.8631
CatBoost	0.5273	0.8878	0.6616	0.9726	0.8065
Tuned XGBoost	0.8804	0.8265	0.8526	0.9754	0.8802

The tuned XGBoost model achieved the strongest overall test performance, particularly in terms of **F1 Score** and **PR-AUC**.

Hyperparameter Tuning

The final XGBoost model was optimized using RandomizedSearchCV.

The tuning objective used **Average Precision**, corresponding to the PR-AUC evaluation approach used in the project.

The search considered:

n_estimators

max_depth

learning_rate

subsample

colsample_bytree

Search configuration:

n_iter = 10

cv = 5

random_state = 42

n_jobs = -1

Best configuration:

n_estimators = 200

max_depth = 6

learning_rate = 0.2

subsample = 1.0

colsample_bytree = 0.8

Best cross-validation PR-AUC during randomized hyperparameter search:

0.849612

Cross-Validation

The tuned XGBoost model was further evaluated using **StratifiedKFold cross-validation with five folds**.

The resulting mean performance was:

Metric	Mean
Precision	0.9120
Recall	0.8122
F1 Score	0.8579
ROC-AUC	0.9819
PR-AUC	0.8476

These results provide an additional assessment of model stability across stratified training folds.

Classification Threshold Optimization

The default classification threshold of 0.50 was not assumed to be optimal.

Because false positives and false negatives have different operational consequences in fraud detection, predicted probabilities were evaluated across multiple thresholds.

The selected threshold was:

0.89

At the optimized threshold:

Metric	Result
Precision	0.9091
Recall	0.8163
F1 Score	0.8602

Confusion matrix:

[[56856, 8],

[18, 80]]

Therefore:

TN = 56,856

FP = 8

FN = 18

TP = 80

The threshold is serialized as:

backend/fraud_threshold.pkl

The FastAPI backend loads this same threshold for deployed predictions.

Final Model

The final deployed model is a **tuned XGBoost classifier**.

Configuration:

XGBoost

n_estimators = 200

max_depth = 6

learning_rate = 0.2

subsample = 1.0

colsample_bytree = 0.8

The model uses class weighting through XGBoost's `scale_pos_weight` during training.

Final classification threshold:

0.89

At the default 0.50 threshold, the tuned model achieved:

Precision = 0.8804

Recall = 0.8265

F1 = 0.8526

ROC-AUC = 0.9754

PR-AUC = 0.8802

At the optimized 0.89 threshold:

Precision = 0.9091

Recall = 0.8163

F1 = 0.8602

Threshold optimization therefore increased precision and F1 score while accepting a small reduction in recall.

Model Serialization

The final deployment artifacts are stored in:

backend/

Trained Model

fraud_model.pkl

Contains the trained tuned XGBoost classifier.

Feature Scaler

scaler.pkl

Contains the StandardScaler fitted on the training data for:

Time

Amount

Classification Threshold

fraud_threshold.pkl

Contains the optimized classification threshold:

0.89

These artifacts allow the backend to load the trained model, preprocessing scaler, and decision threshold without retraining.

Explainability

SHAP

The explainability implementation is contained in:

backend/explain.py

The system uses **SHAP (SHapley Additive exPlanations)** to identify features contributing to an individual prediction.

For each transaction, the system:

1. Generates SHAP values.
2. Associates each value with its feature.
3. Ranks features by absolute contribution.
4. Selects the top five features.
5. Determines whether each contribution increased or reduced fraud risk.

The resulting explanations distinguish between:

increased fraud risk

reduced fraud risk

no meaningful effect

The SHAP explainer is initialized once and reused across requests to avoid recreating it for every prediction.

LLM Explanation Layer

The system optionally uses an OpenAI client to convert the model-derived SHAP information into a concise plain-English explanation.

The LLM receives summarized information including:

- model prediction;
- fraud probability;
- decision threshold;
- top SHAP feature contributions; and
- contribution directions.

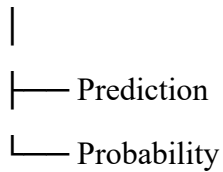
The LLM does **not** receive the raw transaction data for the explanation task.

It does not:

- make the fraud decision;
- calculate the fraud probability;
- override the XGBoost prediction;
- replace SHAP; or
- independently determine whether a transaction is fraudulent.

The architecture is:

XGBoost



SHAP



Top contributing features



LLM





Plain-English explanation

The fraud decision therefore remains model-driven, while the LLM serves as a communication layer.

Graceful LLM Failure

LLM failure does not prevent the core fraud prediction from being returned.

If the LLM is unavailable or fails:

Prediction → still returned

Probability → still returned

SHAP information → still returned

Explanation → predefined fallback

This prevents the optional language-generation service from becoming a dependency for the core fraud-detection decision.

FastAPI Backend

The backend is implemented using **FastAPI**.

Local address:

`http://127.0.0.1:8007`

The backend loads:

`fraud_model.pkl`

`scaler.pkl`

`fraud_threshold.pkl`

API Endpoints

GET /

Returns basic API information.

GET /health

Reports the status of:

- API;
- model;
- scaler; and
- LLM configuration.

POST /predict

Accepts a transaction containing the 30 model features and returns:

- prediction;
 - fraud probability;
 - classification threshold;
 - prediction message;
 - top SHAP features; and
 - plain-English explanation.
-

API Input Validation

Pydantic is used to validate incoming transactions.

The API validates:

- required features;
- numeric input;
- transaction amount;
- transaction time; and
- unexpected additional fields.

Tested invalid-input cases include:

- missing fields;
- negative transaction amount;
- negative transaction time;
- unexpected fields; and
- non-numeric values.

Invalid request structures return HTTP 422 validation responses.

Prediction Pipeline

For an incoming transaction, the backend performs:

Client Request

|



Pydantic Validation

|



Feature Ordering

|



Scale Time + Amount

|



Tuned XGBoost

|

|—————▶ Fraud Probability

|



Threshold = 0.89

|



Binary Prediction

|



SHAP Explanation



LLM / Fallback Explanation



JSON Response

Streamlit Frontend

The interactive frontend is implemented in:

`frontend/app.py`

The Streamlit application communicates with the FastAPI backend rather than performing model inference directly.

Manual Prediction

Users can enter:

Time

Amount

V1–V28

The transaction is submitted to:

`POST /predict`

The application displays:

- predicted class;
- fraud probability;
- decision threshold;
- explanation; and
- top SHAP features.

CSV Prediction

The frontend also supports CSV upload.

The workflow is:

CSV Upload



Transaction Processing



FastAPI /predict



Individual Predictions



Results Table

Fraudulent transactions are highlighted, and the application displays the number of flagged transactions.

Individual results can be expanded to inspect prediction details and explanations.

Deployment Consistency

The deployed API was tested using the same fraudulent transaction used in the training notebook.

Measurement Notebook API

Prediction	1	1
Probability	0.999982	0.999982

Measurement Notebook API

Threshold	0.89	0.89
-----------	------	------

The matching results confirm consistency between:

- notebook preprocessing;
 - serialized scaler;
 - serialized model;
 - probability calculation;
 - classification threshold; and
 - deployed API prediction.
-

Performance Testing

The /predict endpoint was tested using the same fraudulent transaction across 10 requests.

Measurement	Result
Requests completed	10/10
Average response time	62.98 ms
Minimum response time	60.17 ms
Maximum response time	64.83 ms

This provides a response-time baseline for the current local deployment and SHAP-enabled prediction pipeline.

Error Handling

The backend provides controlled handling for invalid requests and explanation-service failures.

The system was tested with:

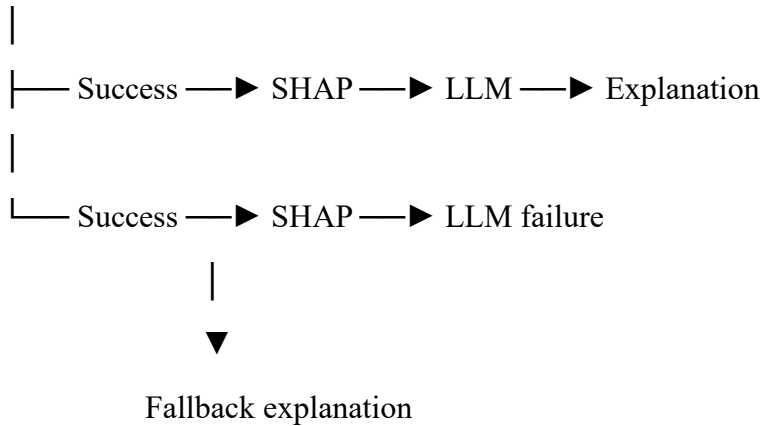
- missing required fields;

- negative transaction amounts;
- negative transaction time;
- unexpected fields; and
- non-numeric values.

For invalid input, the API returns a controlled validation response.

For LLM failure:

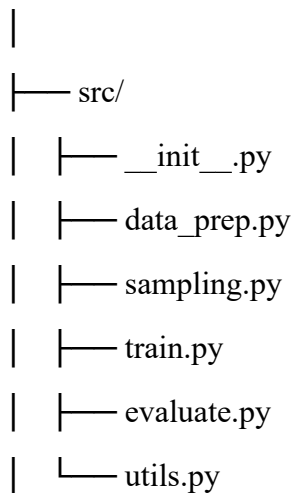
ML prediction



The core prediction therefore remains available even when the optional LLM service fails.

Project Structure

fraud-detection/



```
|
|
|— notebooks/
|   └─ Sylvia_Fraud_Detection_Project.ipynb
|
|
|— backend/
|   └─ main.py
|   └─ explain.py
|   └─ fraud_model.pkl
|   └─ scaler.pkl
|   └─ fraud_threshold.pkl
|
|
|— frontend/
|   └─ app.py
|
|
|— config/
|   └─ config.yaml
|
|
|— data/
|   └─ creditcard.csv
|
|
|— requirements.txt
|— README.md
└─ LICENSE
```

The dataset is excluded from version control through .gitignore and is therefore not included in the public GitHub repository.

Technologies Used

Machine Learning

- Python
- Scikit-learn
- XGBoost
- CatBoost
- imbalanced-learn

Explainability

- SHAP

Backend

- FastAPI
- Uvicorn
- Pydantic
- Joblib

Frontend

- Streamlit
- Requests

Data Analysis and Visualization

- Pandas
- NumPy
- Matplotlib
- Seaborn

LLM

- OpenAI Python SDK

Development

- Jupyter Notebook
- Visual Studio Code

- Conda
 - Git
 - GitHub
-

Installation and Setup

1. Clone the repository

```
git clone https://github.com/Onwuanabile-Sylvia/fraud-detection.git
cd fraud-detection
```

2. Create or activate a Python environment

Install the project dependencies:

```
pip install -r requirements.txt
```

If using Conda:

```
conda create -n fraud_detection python=3.13
```

```
conda activate fraud_detection
```

```
pip install -r requirements.txt
```

3. Start the FastAPI backend

From the project root:

```
uvicorn backend.main:app --reload --port 8007
```

The API will be available at:

```
http://127.0.0.1:8007
```

Interactive Swagger documentation:

```
http://127.0.0.1:8007/docs
```

4. Start the Streamlit frontend

Open another terminal from the project root:

```
python -m streamlit run frontend\app.py
```

The Streamlit application will open in the browser.

Limitations

Dataset limitations

The project uses a historical public dataset and therefore may not represent current real-world fraud patterns.

Anonymized features

The V1–V28 variables are anonymized, limiting direct business interpretation.

Distribution shift

Fraud patterns can change over time. Strong performance on the current dataset does not guarantee equivalent production performance.

Threshold dependence

The selected threshold of 0.89 reflects the current evaluation setup and operational trade-off. Different fraud costs or business requirements could justify a different threshold.

Model monitoring

The current implementation does not include continuous production monitoring or automatic model retraining.

LLM dependency

When enabled, the natural-language explanation layer depends on an external LLM service.

Batch architecture

The Streamlit batch interface currently sends transactions individually to the prediction endpoint rather than using a dedicated batch API endpoint.

Future Improvements

Potential future improvements include:

- real-time model monitoring;
 - data and concept drift detection;
 - automated model retraining;
 - larger and more recent fraud datasets;
 - cost-sensitive threshold optimization;
 - probability calibration;
 - dedicated batch prediction endpoints;
 - API authentication and authorization;
 - database integration;
 - model versioning;
 - cloud deployment;
 - Docker containerization;
 - CI/CD automation;
 - automated API and model tests;
 - production monitoring and observability; and
 - improved fraud-investigation workflows.
-

Conclusion

This project demonstrates the progression from an imbalanced machine learning dataset to a deployable fraud detection application.

The final system combines:

Data Preparation

+

Machine Learning

+

Class Imbalance Handling

+

Hyperparameter Optimization

+

Cross-Validation

+

Threshold Optimization

+

Explainable AI

+

REST API Deployment

+

Interactive Web Interface

The final deployed model is a **tuned XGBoost classifier** using a classification threshold of **0.89**.

The architecture keeps the fraud decision **deterministic and model-driven**, while SHAP provides numerical model explanations and the optional LLM converts those explanations into concise natural language.

Author

Sylvia Obiageli Onwuanabile

Medical Laboratory Scientist | Data Analytics | Machine Learning | Healthcare Analytics

GitHub: <https://github.com/Onwuanabile-Sylvia/>

LinkedIn: <https://www.linkedin.com/in/sylvia-onwuanabile-153b39261/>